

GGTO · CANTV

Sistema de administración de reportes de avería y construcción de puntos ópticos

MANUAL LITERAL · DIAGRAMA RELACIONAL

Diagrama Entidad-Relación — Modelo de datos GGTO

Este manual explica, con palabras sencillas, cómo se conectan las 36 tablas de la base GGTO. Es la versión literal del manual técnico: conserva las mismas secciones y todos los diagramas por dominio, pero los presenta con un lenguaje claro y con los vínculos simplificados para que se lean fácil. Si algo no se pudo verificar, se indica como pendiente de confirmar.

PRODUCTO

GGTO · versión 0.9.0

COLECCIÓN

Manuales literales · Diagrama Relacional

PÚBLICO

Todo público (lenguaje sencillo)

CLIENTE

CANTV C.A. — Central Francisco Salias (Área 4)

DOCUMENTO

docs/Manuales/06-Diagrama-Relacional/01-diagrama-er.md

ACTUALIZACIÓN DE LA FUENTE

29/09/2026

ÍNDICE

1 Empezar en 5 minutos	3
2 Visión general del modelo	4
2.1 36 tablas y 7 dominios	4
2.2 Convención de diagramas (Mermaid erDiagram)	4
3 Diagramas Mermaid por dominio	5
3.1 Vista general de relaciones	5
3.2 Núcleo organizacional y seguridad	6
3.3 Casos (averías y solicitudes)	9
3.4 Despacho y fallas masivas	12
3.5 Gestión técnica (offline)	14
3.6 Insumos (v2)	16
3.7 Soporte y auditoría	18
4 Relaciones principales	19
4.1 Relaciones 1:N	19
4.2 Relaciones N:M	21
4.3 Dependencias de catálogo	21
5 Claves y restricciones	23
5.1 Claves primarias	23
5.2 Claves foráneas	23
5.3 UNIQUE (id_averia)	23
5.4 CHECK de estados	24
5.5 Funciones (generar_id_averia_ref, triggers actualizado_en)	26
6 RLS	27
6.1 caso y despacho	27
6.2 políticas	27
6.3 implicación multi-central (RNF-21)	27
7 Índices	28
7.1 trigram (caso.direccion, sector_direccion.patron)	28
7.2 índices de apoyo	28

Este manual explica, con palabras sencillas, cómo se conectan las 36 tablas de la base GGTO. Es la versión literal del manual técnico: conserva las mismas secciones y todos los diagramas por dominio, pero los presenta con un lenguaje claro y con los vínculos simplificados para que se lean fácil. Si algo no se pudo verificar, se indica como **pendiente de confirmar**.

1 Empezar en 5 minutos

1. **Piensa en un grafo, no en una lista.** Cada tabla es un punto y cada relación es una flecha. Leer el modelo es seguir esas flechas: de una central salen sus sectores, técnicos y casos.
2. **Aprende a leer las flechas.** `||--o{` significa «uno a muchos»: un caso puede tener muchas citas. `||--o|` significa «uno a cero o uno»: un técnico puede tener una cuenta de usuario, o ninguna.
3. **Empieza por el centro.** La tabla `caso` es el corazón. Se conecta con `central`, `sector`, `causa`, `ingesta_lote`, `despacho_caso`, `cita`, `actividad`, `seguimiento` y `notificacion`.
4. **Revisa los tres puentes.** Tres relaciones de muchos a muchos se resuelven con tablas puente: `cuadrilla_tecnico`, `cuadrilla_herramienta` y `despacho_caso`.
5. **Recuerda las reglas del sistema.** La base tiene 36 tablas y ya contiene datos reales (42 casos con datos personales) en un servicio público: el acceso debe restringirse. La tabla número 36, `cuadrilla_sector_dia`, guarda qué sectores atiende cada cuadrilla cada día (ciclo D-66). Telegram y correo no tienen credenciales, así que los envíos quedan PENDIENTES; WhatsApp no existe en la v1; la app móvil Flutter no está desarrollada (Ciclo 8 pospuesto); y el riesgo aceptado D-26 indica que no hay respaldos/PITR hasta migrar de instancia.

2 Visión general del modelo

2.1 36 tablas y 7 dominios

El esquema físico contiene 36 sentencias `CREATE TABLE` y el modelo de origen las representa en siete bloques temáticos:

#	DOMINIO	ENTIDADES DEL DIAGRAMA	Nº
1	Vista general de relaciones	Relaciones entre todas las entidades	—
2	Núcleo organizacional y seguridad	ROL, CENTRAL, SECTOR, SECTOR_DIRECCION, TECNICO, USUARIO, FLOTA, HERRAMIENTA, CUADRILLA, CUADRILLA_TECNICO, CUADRILLA_HERRAMIENTA, DISPOSITIVO_SEGURIDAD, SINCRONIZACION	13
3	Casos: averías y solicitudes	INGESTA_LOTE, CAUSA, CASO, CASO_ESPECIAL, SOLICITANTE, CASO_ESTADO_HIST, SEGUIMIENTO, CITA	8
4	Despacho y fallas masivas	DESPACHO, DESPACHO_CASO, CUADRILLA_SECTOR_DIA, FALLA_MASIVA (más CASO, CUADRILLA, SECTOR y CENTRAL como referencia)	4 propias
5	Gestión técnica	ACTIVIDAD, EVIDENCIA, INCIDENTE, CATALOGO_METODO (más CAUSA, CASO, USUARIO, CUADRILLA, FLOTA y HERRAMIENTA)	4 propias
6	Insumos v2	INSUMO, ORDEN_MATERIAL, ORDEN_MATERIAL_DETALLE, INVENTARIO_MOVIMIENTO	4
7	Soporte y auditoría	CONFIGURACION, AUDITORIA, NOTIFICACION	3

Las entidades definidas con atributos propios en los dominios 2 a 7 suman $13 + 8 + 4 + 4 + 4 + 3 = 36$. Además, cada bloque repite entidades de otros dominios como referencia para dibujar la relación (por ejemplo, `CASO` aparece en los dominios 4, 5 y 7). En total, **36 tablas** distintas.

2.2 Convención de diagramas (Mermaid erDiagram)

El modelo fija las convenciones de tipos, porque **Mermaid no admite paréntesis** en la definición de atributos:

MERMAID	POSTGRESQL REAL
<code>serial</code> / <code>bigserial</code>	<code>serial</code> / <code>bigserial</code>
<code>varchar</code>	<code>varchar(n)</code> según el diccionario
<code>timestamp</code>	<code>timestampz</code>
<code>numeric</code>	<code>numeric(12,2)</code> / <code>numeric(10,7)</code> para GPS
<code>json</code>	<code>jsonb</code>
<code>bool</code>	<code>boolean</code>

Claves: `PK` es clave principal, `FK` clave foránea y `UK` valor único. Las cardinalidades usadas son `||--o{` (uno a muchos, lado opcional) y `||--o|` (uno a cero o uno). Los comentarios entre comillas de los diagramas (por ejemplo `"unico global"` o `"sin solapamiento"`) provienen del modelo y se conservan textualmente; algunos describen reglas que en el script de la base se implementan con `CHECK`, `UNIQUE` o directamente en el servicio.

3 Diagramas Mermaid por dominio

3.1 Vista general de relaciones

Muestra el grafo completo de dependencias entre las 36 tablas. La imagen siguiente es la versión simplificada y legible: solo nombres de tablas y relaciones.

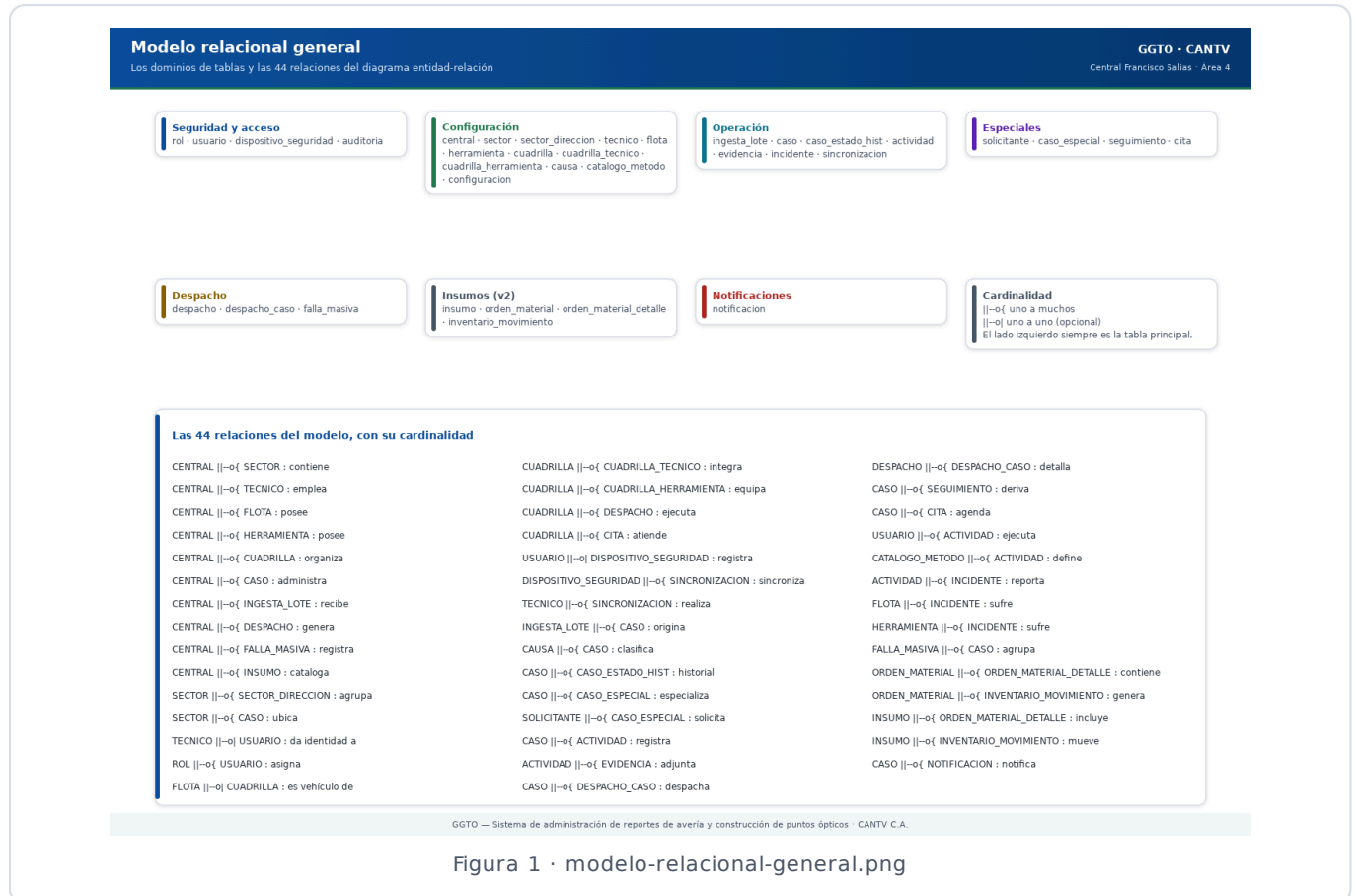


Figura 1 · modelo-relacional-general.png

Observaciones verificadas sobre este grafo:

- La relación «CASO ||--o{ EVIDENCIA» del diagrama **no** es directa en el script de la base: `evidencia` referencia a `actividad` y llega al caso a través de ella. El bloque del dominio 5 sí modela la ruta correcta (`ACTIVIDAD ||--o{ EVIDENCIA`).
- «CENTRAL ||--o{ INSUMO» se cumple solo si `id_central` está informado; en el script de la base la columna admite valores vacíos.
- «FALLA_MASIVA ||--o{ CASO» no tiene clave foránea: la pertenencia se marca con `caso.es_falla_masiva` y la agrupación por concentración se calcula en `app/services/fallas.py`.
- «CUADRILLA_SECTOR_DIA» es la **tabla nueva del ciclo D-66**: guarda qué sectores atiende cada cuadrilla cada día. La combinación (`fecha`, `id_sector`) es **única**, de modo que un sector pertenece como máximo a una cuadrilla por día. El servicio de despacho la usa para repartir los casos y propone una asignación equilibrada cuando no hay ninguna guardada.

3.2 Núcleo organizacional y seguridad

Cubre los catálogos y los recursos de la central, incluidos los de la app de campo (dispositivo y sincronización).

Diagrama 2 (código Mermaid)

```

erDiagram
    ROL {
        serial id_rol PK
        varchar codigo UK "SUPERVISOR|TECNICO|ADMIN"
        varchar nombre
        json permisos
        bool activo
    }
    CENTRAL {
        serial id_central PK
        varchar region
        varchar estado_geografico
        varchar capital_estado
        varchar municipio
        varchar parroquia
        varchar estado_operativo
        varchar distrito
        varchar area
        varchar codigo_central UK "ej. 2324X"
        varchar nombre_central
        bool activa
    }
    SECTOR {
        serial id_sector PK
        int id_central FK
        varchar nombre "Norte 1, Sur 2"
        varchar codigo UK
        varchar descripcion
        int prioridad
        bool activo
    }
    SECTOR_DIRECCION {
        serial id_sector_direccion PK
        int id_sector FK
        varchar patron "VALLE ARRIBA, CONCRESA"
        varchar tipo_coincidencia "CONTIENE|EXACTO|REGEX"
        bool normalizar
        bool activo
    }
    TECNICO {
        serial id_tecnico PK
        int id_central FK
        varchar nombre
        varchar apellido
        varchar cedula UK
        varchar p00 UK
        varchar telefono
        varchar correo
        varchar especialidad
        varchar status
    }
    USUARIO {
        serial id_usuario PK
        varchar p00 UK
        varchar correo UK
        varchar clave_hash
        int id_rol FK
    }

```

Diagrama 2 (código Mermaid) (continuación)

```

    int      id_tecnico FK
    int      id_central FK
    int      intentos_fallidos
    bool     bloqueado
    bool     bloqueo_cliente
    bool     requiere_cambio_clave
    bool     activo
    timestamp ultimo_acceso
}
FLOTA {
    serial    id_flota PK
    int       id_central FK
    varchar   can UK
    varchar   tipo
    varchar   marca
    varchar   modelo
    varchar   placa UK
    varchar   combustible
    varchar   status
    varchar   estado_cauchos
    varchar   estado_fluidos
    varchar   estado_general
}
HERRAMIENTA {
    serial    id_herramienta PK
    int       id_central FK
    varchar   nombre
    varchar   codigo UK
    varchar   estado
    varchar   observacion
}
CUADRILLA {
    serial    id_cuadrilla PK
    int       id_central FK
    varchar   codigo UK
    varchar   nombre
    int       id_flota FK
    bool      es_supervisor "cuadrilla 0"
    bool      activa
}
CUADRILLA_TECNICO {
    int       id_cuadrilla PK
    int       id_tecnico PK
    date      desde PK
    varchar   rol_cuadrilla
    date      hasta
}
CUADRILLA_HERRAMIENTA {
    int       id_cuadrilla PK
    int       id_herramienta PK
    timestamp asignada_en PK
    timestamp devuelta_en
}
DISPOSITIVO_SEGURIDAD {
    bigserial id_dispositivo PK
    varchar   p000 FK
    json      palabras_hash "12 palabras"
}

```

Diagrama 2 (código Mermaid) (continuación)

```

    varchar  clave_privada_ref
    text     documento_cifrado
    bool     bloqueado
    int      version
}
SINCRONIZACION {
    bigserial id_sync PK
    bigint    id_dispositivo FK
    int       id_tecnico FK
    timestamp inicio
    timestamp fin
    int       casos_descargados
    int       actividades_subidas
    int       evidencias_subidas
    varchar   archivo_zip "central+cuadrilla+fecha"
    varchar   estado
    json      detalle_json
}

CENTRAL ||--o{ SECTOR : contiene
SECTOR  ||--o{ SECTOR_DIRECCION : agrupa
CENTRAL ||--o{ TECNICO : emplea
CENTRAL ||--o{ FLOTA : posee
CENTRAL ||--o{ HERRAMIENTA : posee
CENTRAL ||--o{ CUADRILLA : organiza
ROL      ||--o{ USUARIO : asigna
TECNICO  ||--o{ USUARIO : identifica
FLOTA    ||--o{ CUADRILLA : "es vehículo de"
CUADRILLA ||--o{ CUADRILLA_TECNICO : integra
TECNICO  ||--o{ CUADRILLA_TECNICO : pertenece
CUADRILLA ||--o{ CUADRILLA_HERRAMIENTA : equipa
HERRAMIENTA ||--o{ CUADRILLA_HERRAMIENTA : asignada
USUARIO  ||--o{ DISPOSITIVO_SEGURIDAD : registra
DISPOSITIVO_SEGURIDAD ||--o{ SINCRONIZACION : sincroniza
TECNICO  ||--o{ SINCRONIZACION : realiza

```

El comentario "SUPERVISOR|TECNICO|ADMIN" del atributo `ROL.codigo` quedó desactualizado respecto del script de la base: la semilla real inserta **cuatro** roles (`SUPER`, `ADMIN`, `SUPERVISOR`, `TECNICO`). Además, `sector.codigo` y `cuadrilla.codigo` son únicos **por central** (`UNIQUE (id_central, codigo)`), no globales, y `ux_cuadrilla_supervisor` garantiza una sola cuadrilla 0 por central. Los campos `telefono`, `correo`, `cedula`, `nombre` y `apellido` son datos personales: trátalos con cuidado.

3.3 Casos (averías y solicitudes)

Es el núcleo transaccional del sistema.

Diagrama 3 (código Mermaid)

```

erDiagram
    INGESTA_LOTE {
        bigserial id_lote PK
        varchar archivo
        date fecha_archivo
        int id_central FK
        int filas_leidas
        int filas_central
        int casos_nuevos
        int casos_duplicados
        int casos_descartados
        varchar estado
        varchar usuario
    }
    CAUSA {
        serial id_causa PK
        varchar codigo_causa
        varchar subcodigo_causa
        varchar descripcion
        varchar descripcion_subcodigo
        varchar tipo
        bool activo
    }
    CASO {
        bigserial id_caso PK
        int id_central FK
        varchar id_averia UK "unico global"
        varchar origen
        varchar tipo_caso "AVERIA|REPARACION|CONSTRUCCION"
        varchar categoria "RESIDENCIAL|EMPRESA|REFERIDO|GOBIERNO"
        varchar estado_actual
        varchar telefono
        varchar nombre_cliente
        varchar direccion
        int id_sector FK
        int id_causa FK
        bigint id_lote_ingesta FK
        bool en_gestion_supervisor
        bool es_falla_masiva
        varchar olt
        varchar slot
        varchar puerto
        varchar fat
        varchar serial
        varchar plan
        varchar tipo_servicio
        varchar problema_reporte
        varchar ultimo_comentario
        varchar informacion "unifica cols 31,32,52 (D-27)"
        timestamp fecha_reporte
        timestamp fecha_cita
        timestamp fecha_compromiso
    }
    CASO_ESPECIAL {
        serial id_caso_especial PK
        bigint id_caso FK
        int id_solicitante FK
    }

```

Diagrama 3 (código Mermaid) (continuación)

```

    varchar clasificacion "REFERIDO|EMPRESA|GOBIERNO"
    varchar tipo_actividad "REPARACION|CONSTRUCCION"
    varchar prioridad
    bool tiene_id_averia
    text descripcion
    bool requiere_informe
    varchar estado
}
SOLICITANTE {
    serial id_solicitante PK
    varchar unidad
    varchar nombre
    varchar contacto
    varchar canal "TELEGRAM|MCP_IA|MANUAL"
}
CASO_ESTADO_HIST {
    bigserial id_hist PK
    bigint id_caso FK
    varchar estado_anterior
    varchar estado_nuevo
    text motivo
    varchar usuario
    timestamp fecha_hora
}
SEGUIMIENTO {
    bigserial id_seguimiento PK
    bigint id_caso FK
    varchar instancia_destino
    text motivo
    timestamp fecha_envio
    timestamp fecha_retorno
    varchar estado
    varchar usuario
}
CITA {
    bigserial id_cita PK
    bigint id_caso FK
    int id_caso_especial FK
    int id_cuadrilla FK
    timestamp fecha_hora "sin solapamiento"
    varchar tipo "CONTACTO|ATENCION"
    varchar estado
    text observacion
}

CENTRAL ||--o{ INGESTA_LOTE : recibe
INGESTA_LOTE ||--o{ CASO : origina
CENTRAL ||--o{ CASO : administra
SECTOR ||--o{ CASO : ubica
CAUSA ||--o{ CASO : clasifica
CASO ||--o{ CASO_ESPECIAL : especializa
SOLICITANTE ||--o{ CASO_ESPECIAL : solicita
CASO ||--o{ CASO_ESTADO_HIST : historial
CASO ||--o{ SEGUIMIENTO : deriva
CASO ||--o{ CITA : agenda
CASO_ESPECIAL ||--o{ CITA : agenda
CUADRILLA ||--o{ CITA : atiende

```

Notas verificadas: el diagrama declara `CITA.id_cita` como `bigserial`, que es el tipo real; el diccionario de origen lo describía como `serial` (divergencia señalada en el manual del diccionario). La regla «sin solapamiento» del atributo `fecha_hora` se apoya en el índice `ix_cita_cuadrilla_fecha`; en el script de la base **no** existe una restricción `EXCLUDE` (el propio modelo la considera «opcional»). La tabla `caso` guarda teléfono, nombre del cliente y dirección (datos personales).

3.4 Despacho y fallas masivas

Diagrama 4 (código Mermaid)

```

erDiagram
    DESPACHO {
        bigserial id_despacho PK
        int id_central FK
        date fecha
        int id_cuadrilla FK
        varchar estado "BORRADOR|PUBLICADO|CERRADO"
        bool generado_auto
        varchar enviado_canal "TELEGRAM|CORREO"
        timestamp enviado_en
        timestamp reporte_produccion_en "04:00 pm"
        varchar usuario_crea
    }
    DESPACHO_CASO {
        bigserial id_despacho_caso PK
        bigint id_despacho FK
        bigint id_caso FK
        int id_sector FK
        int orden_visita
        varchar tipo_asignacion
        varchar estado
        text observacion
    }
    FALLA_MASIVA {
        bigserial id_falla PK
        int id_central FK
        varchar clave_concentracion
        text descripcion
        timestamp fecha_deteccion
        varchar origen "AUTOMATICA|REPORTE_TECNICO|MCP"
        int id_sector FK
        int id_cuadrilla FK
        varchar estado
        text planificacion
        text reporte_simple
        timestamp planificada_en
    }
    CUADRILLA_SECTOR_DIA {
        bigserial id_asignacion PK
        int id_central FK
        date fecha
        int id_cuadrilla FK
        int id_sector FK
        varchar usuario FK
        timestamp creado_en
    }
    CASO {
        bigserial id_caso PK
        varchar id_averia UK
        bool es_falla_masiva
        int id_sector FK
    }
    CUADRILLA {
        serial id_cuadrilla PK
        varchar codigo UK
        bool es_supervisor
    }

```

Diagrama 4 (código Mermaid) (continuación)

```

SECTOR {
  serial    id_sector PK
  varchar   nombre
}
CENTRAL {
  serial    id_central PK
  varchar   nombre_central
}

CENTRAL ||--o{ DESPACHO : genera
CUADRILLA ||--o{ DESPACHO : ejecuta
DESPACHO ||--o{ DESPACHO_CASO : detalla
CASO ||--o{ DESPACHO_CASO : asignado
SECTOR ||--o{ DESPACHO_CASO : agrupa
CENTRAL ||--o{ CUADRILLA_SECTOR_DIA : programa
CUADRILLA ||--o{ CUADRILLA_SECTOR_DIA : atiende
SECTOR ||--o{ CUADRILLA_SECTOR_DIA : "asignado (único por fecha)"
CENTRAL ||--o{ FALLA_MASIVA : registra
SECTOR ||--o{ FALLA_MASIVA : ubica
CUADRILLA ||--o{ FALLA_MASIVA : atiende
FALLA_MASIVA ||--o{ CASO : agrupa

```

El diagrama ya marca `id_despacho` como `bigserial`, que es el tipo real, a diferencia del diccionario de origen. La relación `FALLA_MASIVA ||--o{ CASO` no tiene clave foránea en el script: se materializa con la bandera `caso.es_falla_masiva` y con la detección por concentración de `app/services/fallas.py`, cuyos parámetros viven en `configuracion` (`fallas.activo`, `fallas.umbral_casos`, `fallas.ventana_horas` y `fallas.campo_concentracion`). El canal `enviado_canal` admite Telegram y correo, pero en producción **aún no hay credenciales**: los envíos quedan PENDIENTES.

La entidad `CUADRILLA_SECTOR_DIA` es la incorporación del ciclo **D-66**. Une tres catálogos (`central`, `cuadrilla` y `sector`) y al usuario que guarda la asignación. Su restricción `UNIQUE (fecha, id_sector)` impide que un mismo sector quede en dos cuadrillas el mismo día. No se conecta con `DESPACHO`: es una lista de trabajo previa que el motor de reparto consulta.

3.5 Gestión técnica (offline)

El título original del documento es «Gestión técnica (app móvil)». Debe precisarse que **la aplicación móvil Flutter no está desarrollada** (Ciclo 8 pospuesto): este bloque describe el modelo previsto para la operación en campo y las tablas existen en el script de la base, pero en la v1 ningún flujo de la API las escribe. WhatsApp tampoco existe en la v1.

Diagrama 5 (código Mermaid)

```

erDiagram
    ACTIVIDAD {
        bigserial id_actividad PK
        bigint id_caso FK
        int id_usuario FK
        int id_cuadrilla FK
        varchar tipo "CONTACTO|CIERRE|ENRUTE|DIFERIDO|INCIDENTE|FALLA_MASIVA"
        varchar resultado
        text reporte_corto
        int id_metodo FK
        int id_causa FK
        timestamp fecha_hora
        numeric latitud
        numeric longitud
        bool sincronizado
    }
    EVIDENCIA {
        bigserial id_evidencia PK
        bigint id_actividad FK
        varchar tipo "POTENCIA|NAVEGACION|DEMO"
        varchar serial_imagen UK "caso+averia+tipo+fecha"
        varchar ruta_local
        varchar ruta_remota
        numeric latitud
        numeric longitud
        timestamp fecha_hora
        bool origen_camara "false = galeria (invalido)"
    }
    INCIDENTE {
        bigserial id_incidente PK
        varchar tipo "FLOTA|HERRAMIENTA"
        int id_flota FK
        int id_herramienta FK
        text descripcion
        bigint id_actividad FK
        varchar estado
    }
    CATALOGO_METODO {
        serial id_metodo PK
        varchar dominio "CIERRE|ENRUTE|DIFERIDO|CONTACTO"
        varchar codigo
        varchar nombre
        bool activo
    }
    CAUSA {
        serial id_causa PK
        varchar codigo_causa
        varchar subcodigo_causa
    }
    CASO {
        bigserial id_caso PK
        varchar id_averia UK
    }
    USUARIO {
        serial id_usuario PK
        varchar p000 UK
    }

```

Diagrama 5 (código Mermaid) (continuación)

```

CUADRILLA {
  serial    id_cuadrilla PK
  varchar   codigo UK
}
FLOTA {
  serial    id_flota PK
  varchar   can UK
}
HERRAMIENTA {
  serial    id_herramienta PK
  varchar   codigo UK
}

CASO      ||--o{ ACTIVIDAD : registra
USUARIO   ||--o{ ACTIVIDAD : ejecuta
CUADRILLA ||--o{ ACTIVIDAD : pertenece
CATALOGO_METODO ||--o{ ACTIVIDAD : define
CAUSA     ||--o{ ACTIVIDAD : justifica
ACTIVIDAD ||--o{ EVIDENCIA : adjunta
ACTIVIDAD ||--o{ INCIDENTE : reporta
FLOTA     ||--o{ INCIDENTE : sufre
HERRAMIENTA ||--o{ INCIDENTE : sufre

```

El bloque del dominio 5 sí incluye `CATALOGO_METODO.dominio = CONTACTO`, que coincide con la regla real y corrige la omisión del diccionario. La regla « `origen_camara` debe ser `true` » se expresa como valor inicial `true` más la validación en el servicio: el script de la base no impone una regla que lo fuerce.

3.6 Insumos (v2)

Corresponde al alcance de la v2 (RF-05); las tablas existen pero no tienen flujo de aplicación en la v1.

Diagrama 6 (código Mermaid)

```

erDiagram
    INSUMO {
        serial id_insumo PK
        int id_central FK
        varchar codigo UK
        varchar nombre
        varchar unidad_medida
        numeric stock_minimo
        bool activo
    }
    ORDEN_MATERIAL {
        bigserial id_orden PK
        bigint id_caso FK
        int id_cuadrilla FK
        varchar solicitante_usuario
        varchar estado
        timestamp fecha
        text observacion
    }
    ORDEN_MATERIAL_DETALLE {
        bigserial id_orden_detalle PK
        bigint id_orden FK
        int id_insumo FK
        numeric cantidad_solicitada
        numeric cantidad_entregada
    }
    INVENTARIO_MOVIMIENTO {
        bigserial id_mov PK
        int id_insumo FK
        varchar tipo "INGRESO|EGRESO|AJUSTE"
        numeric cantidad
        bigint id_orden FK
        int id_tecnico FK
        timestamp fecha
        varchar usuario
    }
    CENTRAL {
        serial id_central PK
    }
    CASO {
        bigserial id_caso PK
    }
    CUADRILLA {
        serial id_cuadrilla PK
    }
    TECNICO {
        serial id_tecnico PK
    }

    CENTRAL ||--o{ INSUMO : cataloga
    ORDEN_MATERIAL ||--o{ ORDEN_MATERIAL_DETALLE : contiene
    INSUMO ||--o{ ORDEN_MATERIAL_DETALLE : incluye
    ORDEN_MATERIAL ||--o{ INVENTARIO_MOVIMIENTO : genera
    INSUMO ||--o{ INVENTARIO_MOVIMIENTO : mueve
    CASO ||--o{ ORDEN_MATERIAL : origina
    CUADRILLA ||--o{ ORDEN_MATERIAL : solicita
    TECNICO ||--o{ INVENTARIO_MOVIMIENTO : recibe

```

Detalle verificado: `inventario_movimiento` tiene además la columna `observacion` que el diagrama no lista, y `orden_material_detalle` restringe cada insumo a una sola aparición por orden (`UNIQUE (id_orden, id_insumo)`).

3.7 Soporte y auditoría

Diagrama 7 (código Mermaid)

```
erDiagram
    CONFIGURACION {
        varchar clave PK
        json valor
        varchar descripcion
        timestamp actualizado_en
    }
    AUDITORIA {
        bigserial id_auditoria PK
        varchar usuario
        varchar accion
        varchar entidad
        varchar id_entidad
        json datos_antes
        json datos_despues
        varchar ip
        timestamp fecha_hora
    }
    NOTIFICACION {
        bigserial id_notificacion PK
        varchar canal "TELEGRAM|CORREO|MCP_IA"
        varchar destinatario
        varchar asunto
        text cuerpo
        bigint id_caso FK
        varchar estado
        timestamp enviado_en
        int intentos
        timestamp proximo_intento
    }
    CASO {
        bigserial id_caso PK
        varchar id_averia UK
    }

    CASO ||--o{ NOTIFICACION : notifica
```

Detalle verificado: `auditoria.ip` es de tipo `inet`, no `varchar`; tanto `auditoria` como `configuracion` carecen de clave foránea; y `notificacion` incorpora la columna `error` que el diagrama no lista. En producción, los canales Telegram y correo aún no tienen credenciales: las notificaciones quedan en estado `PENDIENTE` (patrón `outbox`, `app/services/outbox.py`).

4 Relaciones principales

4.1 Relaciones 1:N

Relaciones uno-a-muchos verificadas contra las claves foráneas del script de la base:

PADRE	HIJO	CLAVE FORÁNEA	CARDINALIDAD
central	sector	sector.id_central	1:N obligatoria
central	tecnico	tecnico.id_central	1:N obligatoria
central	flota	flota.id_central	1:N obligatoria
central	herramienta	herramienta.id_central	1:N obligatoria
central	cuadrilla	cuadrilla.id_central	1:N obligatoria
central	caso	caso.id_central	1:N obligatoria
central	ingesta_lote	ingesta_lote.id_central	1:N opcional
central	despacho	despacho.id_central	1:N obligatoria
central	falla_masiva	falla_masiva.id_central	1:N obligatoria
central	insumo	insumo.id_central	1:N opcional
central	cuadrilla_sector_dia	cuadrilla_sector_dia.id_central	1:N obligatoria
sector	sector_direccion	sector_direccion.id_sector	1:N con borrado en cascada
sector	caso	caso.id_sector	1:N opcional
sector	despacho_caso	despacho_caso.id_sector	1:N opcional
sector	cuadrilla_sector_dia	cuadrilla_sector_dia.id_sector	1:N con borrado en cascada
ingesta_lote	caso	caso.id_lote_ingesta	1:N opcional
causa	caso	caso.id_causa	1:N opcional
causa	actividad	actividad.id_causa	1:N opcional
catalogo_metodo	actividad	actividad.id_metodo	1:N opcional
flota	cuadrilla	cuadrilla.id_flota	1:N opcional
cuadrilla	despacho	despacho.id_cuadrilla	1:N obligatoria
cuadrilla	cita	cita.id_cuadrilla	1:N opcional
cuadrilla	actividad	actividad.id_cuadrilla	1:N opcional
cuadrilla	falla_masiva	falla_masiva.id_cuadrilla	1:N opcional
cuadrilla	orden_material	orden_material.id_cuadrilla	1:N opcional
cuadrilla	cuadrilla_sector_dia	cuadrilla_sector_dia.id_cuadrilla	1:N con borrado en cascada
caso	caso_estado_hist	caso_estado_hist.id_caso	1:N con borrado en cascada
caso	seguimiento	seguimiento.id_caso	1:N con borrado en cascada
caso	cita	cita.id_caso	1:N con borrado en cascada
caso	despacho_caso	despacho_caso.id_caso	1:N con borrado en cascada
caso	actividad	actividad.id_caso	1:N con borrado en cascada
caso	notificacion	notificacion.id_caso	1:N con borrado que deja el campo vacío
caso	orden_material	orden_material.id_caso	1:N opcional
caso_especial	cita	cita.id_caso_especial	1:N con borrado en cascada
solicitante	caso_especial	caso_especial.id_solicitante	1:N opcional
despacho	despacho_caso	despacho_caso.id_despacho	1:N con borrado en cascada

PADRE	HIJO	CLAVE FORÁNEA	CARDINALIDAD
actividad	evidencia	evidencia.id_actividad	1:N con borrado en cascada
actividad	incidente	incidente.id_actividad	1:N con borrado que deja el campo vacío
usuario	actividad	actividad.id_usuario	1:N opcional
usuario	orden_material	orden_material.solicitante_usuario (apunta a p00)	1:N opcional
usuario	cuadrilla_sector_dia	cuadrilla_sector_dia.usuario (apunta a p00)	1:N opcional
flota	incidente	incidente.id_flota	1:N opcional
herramienta	incidente	incidente.id_herramienta	1:N opcional
dispositivo_seguridad	sincronizacion	sincronizacion.id_dispositivo	1:N con borrado que deja el campo vacío
tecnico	sincronizacion	sincronizacion.id_tecnico	1:N opcional
tecnico	inventario_movimiento	inventario_movimiento.id_tecnico	1:N opcional
insumo	orden_material_detalle	orden_material_detalle.id_insumo	1:N obligatoria
insumo	inventario_movimiento	inventario_movimiento.id_insumo	1:N obligatoria
orden_material	orden_material_detalle	orden_material_detalle.id_orden	1:N con borrado en cascada
orden_material	inventario_movimiento	inventario_movimiento.id_orden	1:N opcional

4.2 Relaciones N:M

Las tres relaciones muchos-a-muchos se resuelven con **tablas puente** que tienen clave principal compuesta:

TABLA PUENTE	EXTREMOS	CLAVE PRINCIPAL COMPUESTA	ATRIBUTOS PROPIOS
cuadrilla_tecnico	cuadrilla ↔ tecnico	(id_cuadrilla, id_tecnico, desde)	rol_cuadrilla, hasta
cuadrilla_herramienta	cuadrilla ↔ herramienta	(id_cuadrilla, id_herramienta, asignada_en)	devuelta_en
despacho_caso	despacho ↔ caso	id_despacho_caso (subrogada) + UNIQUE (id_despacho, id_caso)	id_sector, orden_visita, tipo_asignacion, estado, observacion

La pertenencia a cuadrilla es **histórica y con vigencia** (desde / hasta), de modo que un técnico puede cambiar de cuadrilla sin perder el registro anterior. La asignación de herramientas usa la marca temporal como parte de la clave para permitir múltiples asignaciones sucesivas de la misma herramienta a la misma cuadrilla.

4.3 Dependencias de catálogo

Catálogos y su consumo:

CATÁLOGO	CONSUMIDO POR	CLAVE FORÁNEA	NOTAS
rol	usuario.id_rol	Sí, obligatoria	4 roles cargados, incluido <code>SUPER</code> con permiso total.
central	11 tablas (<code>sector</code> , <code>tecnico</code> , <code>flota</code> , <code>herramienta</code> , <code>cuadrilla</code> , <code>caso</code> , <code>ingesta_lote</code> , <code>despacho</code> , <code>cuadrilla_sector_dia</code> , <code>falla_masiva</code> , <code>insumo</code>)	Sí	Base del filtro de ingesta y del alcance multi-central.
sector	<code>caso</code> , <code>sector_direccion</code> , <code>despacho_caso</code> , <code>cuadrilla_sector_dia</code> , <code>falla_masiva</code>	Sí	Asignación de sector por coincidencia de dirección; <code>cuadrilla_sector_dia</code> guarda qué cuadrilla lo atiende cada día.
causa	<code>caso.id_causa</code> , <code>actividad.id_causa</code>	Sí	Catálogo administrable; hoy tiene 0 filas.
catalogo_metodo	<code>actividad.id_metodo</code>	Sí	9 métodos cargados.
configuracion	Ninguna clave foránea; la leen los servicios	No	Parámetros en JSON; se actualiza por <code>PUT /api/v1/configuracion/{clave}</code> .
usuario	<code>creado_por</code> , <code>usuario</code> , <code>creado_por</code> de <code>cita</code> , <code>solicitante_usuario</code> , <code>usuario_crea</code>	Parcial	<code>caso.creado_por</code> referencia <code>usuario(p00)</code> ; otras columnas con el mismo nombre son <code>varchar(20)</code> sin clave foránea.

5 Claves y restricciones

5.1 Claves primarias

TIPO	TABLAS	OBSERVACIÓN
PK simple serial	rol, causa, catalogo_metodo, central, sector, sector_direccion, tecnico, usuario, flota, herramienta, cuadrilla, solicitante, caso_especial, insumo	Catálogos y entidades maestras.
PK simple bigserial	ingesta_lote, caso, caso_estado_hist, seguimiento, cita, despacho, despacho_caso, cuadrilla_sector_dia, falla_masiva, actividad, evidencia, incidente, dispositivo_seguridad, sincronizacion, orden_material, orden_material_detalle, inventario_movimiento, notificacion, auditoria	Tablas transaccionales de alto volumen.
PK compuesta	cuadrilla_tecnico (id_cuadrilla, id_tecnico, desde), cuadrilla_herramienta (id_cuadrilla, id_herramienta, asignada_en)	Dos tablas puente.
PK natural varchar	configuracion.clave	La clave es el nombre del parámetro.

5.2 Claves foráneas

Políticas de borrado observadas en el script de la base:

POLÍTICA	EJEMPLOS
Borrado en cascada (al borrar el padre se borran los hijos)	sector_direccion.id_sector, caso_estado_hist.id_caso, seguimiento.id_caso, cita.id_caso, despacho_caso.id_despacho e id_caso, actividad.id_caso, evidencia.id_actividad, caso_especial.id_caso, orden_material_detalle.id_orden, dispositivo_seguridad.p00, cuadrilla_sector_dia.id_cuadrilla e id_sector
Borrado que deja el campo vacío (el hijo sobrevive)	incidente.id_actividad, sincronizacion.id_dispositivo, notificacion.id_caso
Sin política de borrado (restrictiva por defecto)	La mayoría de las claves foráneas hacia catálogos (central, rol, sector, causa, catalogo_metodo, tecnico, flota, herramienta, cuadrilla)

Consecuencia operativa: no se puede eliminar una **central** que aún tenga sectores, técnicos, casos o despachos (la clave foránea lo impide); en cambio, borrar un **caso** arrastra en cascada su historial de estados, seguimientos, citas y asignaciones de despacho.

5.3 UNIQUE (id_averia)

La unicidad **global** de **caso.id_averia** es la clave para no duplicar casos (RF-22):

- Definición: `id_averia varchar(30) NOT NULL UNIQUE`.
- Formato sintético para casos sin incidencia de origen: `REF-<CÓDIGO_CENTRAL>-<NNNNNN>` (decisión D-23).
- Generación: función `generar_id_averia_ref(p_id_central integer)`, que lee `central.codigo_central`, lanza un error si la central no existe, pasa el código a mayúsculas y rellena con ceros a la izquierda hasta 6 dígitos.
- Secuencia de apoyo: `seq_caso_ref`.
- Ejemplo documentado: `REF-2324X-000001`.

Otras restricciones únicas destacadas: `rol.codigo`, `central.codigo_central`, `tecnico.p00`, `tecnico.cedula`, `usuario.p00`, `usuario.correo`, `flota.can`, `flota.placa`, `herramienta.codigo`, `insumo.codigo`, `evidencia.serial_imagen`; compuestas `sector(id_central, codigo)`, `cuadrilla(id_central, codigo)`, `sector_direccion(id_sector, patron)`, `causa(codigo_causa, subcodigo_causa)`, `catalogo_metodo(dominio, codigo)`, `despacho(fecha, id_cuadrilla)`, `despacho_caso(id_despacho, id_caso)`, `cuadrilla_sector_dia(fecha, id_sector)`, `orden_material_detalle(id_orden, id_insumo)`. El índice único parcial `ux_cuadrilla_supervisor` implementa «una sola cuadrilla 0 por central».

5.4 CHECK de estados

Los enumerados se modelan con listas cerradas `CHECK` (no con el tipo `ENUM`). La columna «Valores» reproduce el orden del script:

TABLA.COLUMNNA	VALORES PERMITIDOS
catalogo_metodo.dominio	CIERRE , ENRUTE , DIFERIDO , CONTACTO
sector_direccion.tipo_coincidencia	CONTIENE , EXACTO , REGEX
tecnico.status	ACTIVO , INACTIVO , VACACIONES , SUSPENDIDO
flota.status	DISPONIBLE , EN_RUTA , MANTENIMIENTO , FUERA_SERVICIO
herramienta.estado	DISPONIBLE , ASIGNADA , AVERIADA , PERDIDA
cuadrilla_tecnico.rol_cuadrilla	REPARADOR_PRINCIPAL , AYUDANTE , SUPERVISOR
ingesta_lote.estado	PROCESANDO , OK , ERROR
caso.origen	INGESTA_CSV , MANUAL , TELEGRAM , MCP_IA
caso.tipo_caso	AVERIA , REPARACION , CONSTRUCCION
caso.categoria	RESIDENCIAL , EMPRESA , REFERIDO , GOBIERNO
caso.estado_actual	NUEVO , ASIGNADO , CONTACTADO , CITADO , DIFERIDO , EN_GESTION , ENRUTADO , CERRADO , CANCELADO
solicitante.canal	TELEGRAM , MCP_IA , MANUAL , CORREO
caso_especial.clasificacion	REFERIDO , EMPRESA , GOBIERNO
caso_especial.tipo_actividad	REPARACION , CONSTRUCCION
caso_especial.prioridad	ALTA , MEDIA , BAJA
caso_especial.estado	ABIERTO , EN_PROCESO , ATENDIDO , CERRADO
seguimiento.estado	EN_COLA , RESUELTO , DEVUELTO
cita.tipo	CONTACTO , ATENCION
cita.estado	PROPUESTA , CONFIRMADA , CUMPLIDA , REPROGRAMADA , DIFERIDA , CANCELADA
despacho.estado	BORRADOR , PUBLICADO , CERRADO
despacho.enviado_canal	TELEGRAM , CORREO
despacho_caso.tipo_asignacion	REPARACION , CONSTRUCCION , REFERIDO , EMPRESA , FALLA_MASIVA
despacho_caso.estado	ASIGNADO , GESTIONADO , CERRADO , CITADO , DIFERIDO
falla_masiva.origen	AUTOMATICA , REPORTE_TECNICO , MCP
falla_masiva.estado	DETECTADA , PLANIFICADA , ATENDIDA , CERRADA
actividad.tipo	CONTACTO , CIERRE , ENRUTE , DIFERIDO , INCIDENTE , FALLA_MASIVA
actividad.resultado	CONTACTADO , CERRADO , ENRUTADO , DIFERIDO
evidencia.tipo	POTENCIA , NAVEGACION , DEMO
incidente.tipo	FLOTA , HERRAMIENTA
incidente.estado	REPORTADO , EN_REVISION , RESUELTO
sincronizacion.estado	EN_PROCESO , OK , ERROR
orden_material.estado	SOLICITADA , APROBADA , ENTREGADA , RECHAZADA
inventario_movimiento.tipo	INGRESO , EGRESO , AJUSTE
notificacion.canal	TELEGRAM , CORREO , MCP_IA
notificacion.estado	PENDIENTE , ENVIADO , FALLIDO

Restricciones de comprobación que no son listas de estados:

REGLA	EN PALABRAS
<code>cita: id_caso IS NOT NULL OR id_caso_especial IS NOT NULL</code>	Una cita debe apuntar a un caso o a un caso especial.
<code>incidente: id_flota IS NOT NULL OR id_herramienta IS NOT NULL</code>	Un incidente debe referirse a un vehículo o a una herramienta.
<code>orden_material_detalle.cantidad_solicitada > 0</code>	La cantidad pedida debe ser positiva.
<code>inventario_movimiento.cantidad > 0</code>	La cantidad del movimiento debe ser positiva.

5.5 Funciones (generar_id_averia_ref, triggers actualizado_en)

Funciones definidas en el esquema:

FUNCIÓN	TIPO	PROPÓSITO
<code>set_actualizado_en()</code>	plpgsql, disparador	Asigna la fecha y hora actual a <code>actualizado_en</code> en cada <code>UPDATE</code> .
<code>generar_id_averia_ref(p_id_central integer)</code>	plpgsql	Devuelve <code>REF-<CÓDIGO>-<NNNNNN></code> para casos sin incidencia.
<code>app_central_actual()</code>	sql STABLE	Lee el parámetro de sesión <code>app.id_central</code> y lo convierte a entero.

Disparadores `actualizado_en`: un bloque `DO` recorre la lista de 14 tablas (`central`, `sector`, `tecnico`, `usuario`, `flota`, `herramienta`, `cuadrilla`, `caso`, `caso_especial`, `despacho`, `falla_masiva`, `insumo`, `orden_material`, `configuracion`) y crea `trg_<tabla>_actualizado BEFORE UPDATE ... FOR EACH ROW`. Hallazgo verificado: `dispositivo_seguridad` declara la columna `actualizado_en` pero **no** figura en la lista, por lo que su marca no se refresca automáticamente. La secuencia `seq_caso_ref` alimenta la numeración de los identificadores `REF-`.

6 RLS

6.1 caso y despacho

El script habilita Row Level Security (RLS, seguridad por filas) con forzado de política en dos tablas:

```
ALTER TABLE caso      ENABLE ROW LEVEL SECURITY;
ALTER TABLE caso      FORCE  ROW LEVEL SECURITY;
ALTER TABLE despacho  ENABLE ROW LEVEL SECURITY;
ALTER TABLE despacho  FORCE  ROW LEVEL SECURITY;
```

`FORCE ROW LEVEL SECURITY` hace que las políticas se apliquen también al propietario de la tabla, de modo que no basta con ser dueño del objeto para saltarlas.

6.2 políticas

Se definen dos políticas, una por tabla, con la misma expresión:

POLÍTICA	TABLA	EXPRESIÓN
p_caso_central	caso	<code>app_central_actual() IS NULL OR id_central = app_central_actual()</code>
p_despacho_central	despacho	Expresión idéntica

La función auxiliar `app_central_actual()` devuelve `null` cuando el parámetro de sesión `app.id_central` no está definido. En ese caso la política **permite todo** (modo compatibilidad declarado explícitamente en el script). El propio comentario advierte que, antes de producción, debe eliminarse la cláusula `app_central_actual() IS NULL` para pasar a un comportamiento de «denegar por defecto».

Estado real de la aplicación: una búsqueda en el código Python **no encuentra ninguna sentencia** `SET LOCAL app.id_central` ni una escritura equivalente. Es decir, en la v1 el backend no fija el alcance de central en la conexión y, por tanto, las políticas RLS no restringen filas: el aislamiento multi-central depende hoy de `usuario.id_central` en la capa de servicio, no de la base. Debe considerarse **pendiente de confirmar** el punto donde se fijará ese parámetro (sesión por petición) o la eliminación de la cláusula permisiva.

6.3 implicación multi-central (RNF-21)

RNF-21 exige «matriz rol × módulo × acción», alcance por `usuario.id_central`, RLS como defensa en profundidad y **pruebas negativas** (la central A no accede a datos de la central B). La situación verificada es:

ELEMENTO DE RNF-21	ESTADO VERIFICADO
Alcance por <code>usuario.id_central</code>	La columna existe y está mapeada en los modelos de Python.
RLS en PostgreSQL	Definida en <code>caso</code> y <code>despacho</code> , pero no activada por la aplicación (sin <code>SET LOCAL app.id_central</code>).
Defensa en profundidad	Parcial: la extensión al resto de tablas con <code>id_central</code> (<code>sector</code> , <code>tecnico</code> , <code>flota</code> , <code>cuadrilla</code> , <code>insumo</code>) está prevista para la Fase 3.
Pruebas negativas entre centrales	pendiente de confirmar ; no se localizaron pruebas de acceso denegado cruzado.

Implicación práctica: mientras exista una sola central cargada (2324X FRANCISCO SALIAS), el comportamiento funcional es correcto; al incorporar una segunda central, la ausencia de `app.id_central` en la conexión convierte las políticas en no restrictivas y el aislamiento recae por completo en la capa de servicio. Recuerda que la base ya contiene datos reales (42 casos con datos personales) y el servicio es público: **el acceso debe restringirse**.

7 Índices

7.1 trigram (caso.direccion, sector_direccion.patron)

La asignación de sector por coincidencia de texto (RF-23/D-33) se apoya en la extensión `pg_trgm` y dos índices GIN con la clase de operadores `gin_trgm_ops`:

ÍNDICE	TABLA.COLUMNA	DEFINICIÓN
<code>ix_caso_direccion_trgm</code>	<code>caso.direccion</code>	<code>USING gin (direccion gin_trgm_ops)</code>
<code>ix_sector_patron_trgm</code>	<code>sector_direccion.patron</code>	<code>USING gin (patron gin_trgm_ops)</code>

Estos índices habilitan búsquedas difusas o similares (`%`, `similarity`) sobre direcciones normalizadas y permiten que la asignación de sector escale con el volumen de casos. El campo `sector_direccion.normalizar` indica que la comparación se hace sin acentos ni mayúsculas, y `tipo_coincidencia` admite `CONTIENE`, `EXACTO` o `REGEX`.

7.2 índices de apoyo

ÍNDICE	TABLA (COLUMNAS)	PROPÓSITO
<code>ux_cuadrilla_supervisor</code>	<code>cuadrilla (id_central) WHERE es_supervisor</code>	Único parcial: una cuadrilla 0 por central
<code>ix_caso_central</code>	<code>caso (id_central)</code>	Filtro por central (RLS y consultas)
<code>ix_caso_sector</code>	<code>caso (id_sector)</code>	Casos por sector
<code>ix_caso_telefono</code>	<code>caso (telefono)</code>	Búsqueda en el panel
<code>ix_caso_estado</code>	<code>caso (estado_actual)</code>	Filtro por estado
<code>ix_caso_fecha_reporte</code>	<code>caso (fecha_reporte)</code>	Monitoreo por fecha
<code>ix_caso_gestion_supervisor</code>	<code>caso (en_gestion_supervisor)</code>	Cola de gestión (cuadrilla 0)
<code>ix_caso_falla_masiva</code>	<code>caso (es_falla_masiva)</code>	Casos agrupados en fallas
<code>ix_caso_lote</code>	<code>caso (id_lote_ingesta)</code>	Trazabilidad de la ingesta
<code>ix_despacho_caso_caso</code>	<code>despacho_caso (id_caso)</code>	Casos de un despacho
<code>ix_despacho_caso_despacho</code>	<code>despacho_caso (id_despacho)</code>	Detalle por despacho
<code>ix_cuadrilla_sector_dia_fecha</code>	<code>cuadrilla_sector_dia (fecha, id_cuadrilla)</code>	Asignación de sectores del día (D-66)
<code>ix_actividad_caso</code>	<code>actividad (id_caso)</code>	Actividades de un caso
<code>ix_actividad_usuario</code>	<code>actividad (id_usuario)</code>	Actividades por técnico
<code>ix_evidencia_actividad</code>	<code>evidencia (id_actividad)</code>	Evidencias de una actividad
<code>ix_cita_cuadrilla_fecha</code>	<code>cita (id_cuadrilla, fecha_hora)</code>	Soporte de la regla «sin solapamiento» (RF-12)
<code>ix_seguimiento_caso</code>	<code>seguimiento (id_caso)</code>	Seguimientos de un caso
<code>ix_sector_direccion_sector</code>	<code>sector_direccion (id_sector)</code>	Patrones de un sector
<code>ix_notificacion_caso</code>	<code>notificacion (id_caso)</code>	Notificaciones de un caso (outbox)
<code>ix_auditoria_fecha</code>	<code>auditoria (fecha_hora)</code>	Consulta de bitácora por fecha

Total de objetos de índice: **22** (21 `CREATE INDEX` más `ux_cuadrilla_supervisor`), de los cuales 2 son trigram y 1 es único parcial. Observación: varias claves foráneas de catálogos pequeños (`causa`, `rol`, `catalogo_metodo`) no tienen índice dedicado; su volumen es bajo y el planificador puede resolverlas con exploraciones secuenciales. Los índices que cubren las consultas de mayor volumen (`caso`, `despacho_caso`, `actividad`, `notificacion`) sí están presentes. Es **pendiente de confirmar**, con `EXPLAIN ANALYZE` sobre datos reales, si falta algún índice compuesto para los reportes de monitoreo (`app/services/monitoreo.py`).